

Fringe Lights: Colored Penumbra in Glimpse

Emanuel Schrade

Animal Logic
Australia
emanuels@al.com.au

Basile Fraboni

Animal Logic
Canada
basilef@al.com.au

Thibault Vergne

Animal Logic
Canada
thibaultv@al.com.au

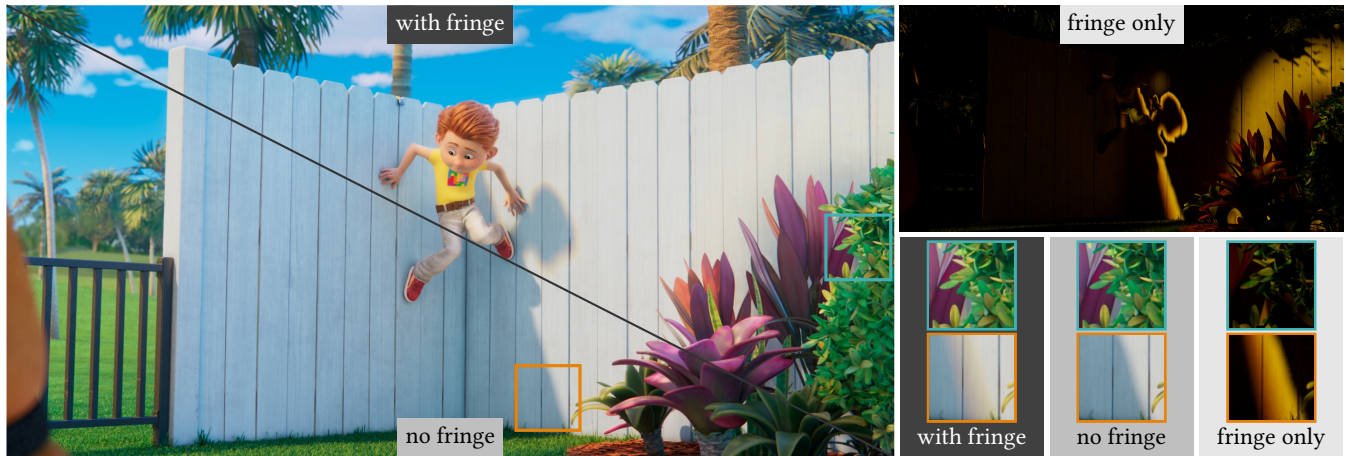


Figure 1: A frame from Netflix’s Leo using Glimpse fringe lights. Tracing at most one additional shadow ray during light source sampling allows to color the penumbra without affecting the rest of the lighting. © 2023 Netflix US, LLC. All Rights Reserved.

Abstract

The recent popularity of stylized rendering calls for new approaches to help artists produce the desired look. Although some effects can be added to the rendered frames in compositing, this requires manual work at the end of the pipeline, leaving preceding stages unclear as to the impact on the final image. In this paper, we present our implementation of *fringe lights* in our production path tracer, *Glimpse*, for Netflix’s Leo. Fringe lights allow lighting artists to easily modify the appearance of the penumbra, leaving both the unoccluded and fully shadowed parts of the scene unaffected. The soft transition between these two regions can be estimated by tracing at most one additional shadow ray towards the light source, allowing separate illumination at a minimal computational overhead. We extended our light sampling and evaluation for fringe lights to account for three different shading regions and modify the light contribution accordingly. Our method offers artists the freedom to stylize the penumbra and match the desired look of the show with direct visual feedback at every stage of the pipeline.

CCS Concepts

• Computing methodologies → Non-photorealistic rendering; Ray tracing.

Keywords

Non-photorealistic rendering, shadow stylization, ray tracing

ACM Reference Format:

Emanuel Schrade, Basile Fraboni, and Thibault Vergne. 2024. Fringe Lights: Colored Penumbra in Glimpse. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3697998>

1 Introduction and Related work

Controlling the look of the penumbra can be valuable for artists to influence the lighting of computer generated images and enhance immersion by adding visual cues about the surrounding scene. In reality, numerous effects contribute to the appearance of shadows, such as indirect light bouncing off nearby surfaces or chromatic volumetric scattering, including subsurface scattering. Additionally, the viewer’s visual system reacts spatially and temporally to the colors surrounding its fixation point, meaning that the observer adapts to nearby and previously seen colors. This phenomenon, known as (local / temporal) chromatic adaptation or color constancy, affects how a color is perceived by the human eye [Foster 2011]. In our context, this can result in a perceived coloration of shadow edges (even achromatic ones). While human visual adaptation and indirect lighting occur both in reality and when generating and viewing a digital image on a screen, it can be desirable for artists

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA Technical Communications '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1140-4/24/12

<https://doi.org/10.1145/3681758.3697998>

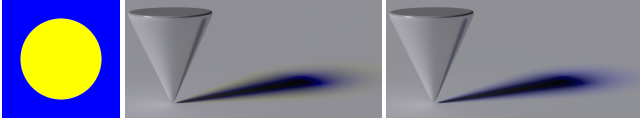


Figure 2: Traditional light maps (left) for adding a blue shadow fringe introduce yellow artifacts (middle) due to normalization. Our fringe lights are free of this artifact (right).

to exaggerate the effect to convey a specific feel of the virtual environment, especially in a non-photorealistic context. For this reason, several artist workflows have been proposed to create the often sought-after effect of *colored shadow fringes*.

A straightforward approach could be to color shadow areas during compositing using shadow mattes. However, this means that earlier stages in the production pipeline are unaware of the impact on the final image.

A recent [Blender plugin](#) provides a surface shader that is applied to the shadow receiver. The shader adjusts its color based on the distance between the shaded point and its shadow caster. This approach leverages the correlation between penumbra size and distance to the shadow caster. As the material is applied to the shadow receiver, it is impossible to apply the effect to a single light source, or to modify it from one light source to another. Furthermore, even though there is a correlation between penumbra size and distance to the occluder, changing the size or position of the light source changes the size of the penumbra and hence, the parameters of all shadow receiving materials need to be updated. In summary, this clever idea is impractical in a production context.

A different approach adds light sources to the scene that purposely illuminate the penumbra region [[Brejon 2020](#), Chapter 7]. A primary light source defines the general lighting, and a secondary, less bright, colored light source illuminates areas that are occluded from the primary light. In the case of area lights, the secondary light can be a scaled up version of the primary light that illuminates part of the penumbra region of the primary light. One drawback of this method is that the secondary light source, which is intended to influence mainly the penumbra region, also affects the lighting of other parts of the scene within its extent. This limits the magnitude of the effect of this approach and generally, the interplay between colored light sources is nontrivial [[Hasenfratz et al. 2003](#)] which makes it hard for artists to control.

In early experiments with colored shadow fringes, artists created light maps consisting of two distinct colored regions with the following motivation (see [Figure 2](#)): as only a portion of the light source is visible in the penumbra region, coloring this part of the light source through a light map would color the penumbra region. This approach combines the two lights from the previous technique into one, where the inner part of the light map represents a "smaller" light source with smaller penumbra region, and the outer part represents the secondary (colored) light source that illuminates the shadowed region. To prevent a color shift in the scene lighting, the contribution of the fully visible light needs to be neutral. This restricts choices for the fringe color and magnitude of the effect, as the light map must integrate to white. This normalization introduces an unwanted coloring in the complementary color of the

chosen fringe color (yellow in [Figure 2](#), middle). As this approach is implemented in a single light source, it is not easy to isolate the penumbra for further adjustments during compositing. Nevertheless, it served as an inspiration for our integration of shadow fringes into our renderer Glimpse, which doesn't suffer from color artifacts ([Figure 2](#), right) and allows to render the colored penumbra to a separate light group for adjustments during compositing.

Two approaches that look similar to ours are shown in a recent blog post discussing Pixar's Luca [[Pedersen and Sisson 2023](#)] and in a demonstration by [Max Liani](#). Both have been developed independently during the same period as our fringe lights and no technical details are published.

2 Fringe lights in glimpse

In this section, we describe our USD-compatible fringe lights, and present a novel stochastic algorithm for sampling such lights. Our implementation in Glimpse is inspired by the idea of splitting light sources into two. We define a large, [outer](#) light source that controls the extent of the colored fringe region and a small, [inner](#) light source, that is used to determine the uncolored shadow extent.

Fringe region. This region is controlled through a configurable scaling factor m , which determines the parameters of the outer light relative to the inner light. For area lights, the scaling factor scales the light radius: $r_o = r_i \cdot m$. For distant and spot lights, the scaling factor applies to the light emission angle: $\theta_o = \theta_i \cdot m$. Varying the scaling factor changes the overlap of the (uncolored) penumbra and (colored) fringe regions. For $m = 1$, the fringe covers the same area as the penumbra. For $m > 1$, the fringe affects an area larger than the penumbra, hence, the colored fringe reaches further into the core shadow. For $m < 1$, the fringe would affect a smaller area than the penumbra. However, due to the smaller extent of the fringe, the overall brightness gradient might look unrealistic (see [Figure 5](#)). While we don't restrict values of $m \geq 1$, this is the more useful case for us.

Sampling fringe lights. In [Figure 3](#), we illustrate our stochastic sampling routine for fringe area lights, further summarized in [Algorithm 1](#). As previously mentioned, the idea is to split the light into two parts depicted in orange and cyan.

To generate a sample on the light and determine its contribution, we first sample a random position s_o on the [outer](#) light as we would if this was a usual area light. We trace a first shadow ray from the shading point x to the light sample s_o . If the sample is occluded, as in case (a), the surface point is not lit, and similarly to a regular light, we return a contribution of zero. If, however, s_o is visible from x , we draw a second random sample s_i , this time from the [inner](#) light source and trace a shadow ray to determine the visibility from x . If s_i is occluded, like in case (b), the shaded point is considered partially visible and thus receives its contribution from the fringe color. If s_i is visible, like in case (c), the shaded point is considered fully visible and receives its contribution from the regular light color. Note that in a variant, the latter case (c) could be treated as shadowed by setting the light color to zero. This would result in a fringe-only light that illuminates only the fringe region. We could optionally change the color of the shadowed area if desired, but that is orthogonal to our method which affects only the penumbra.

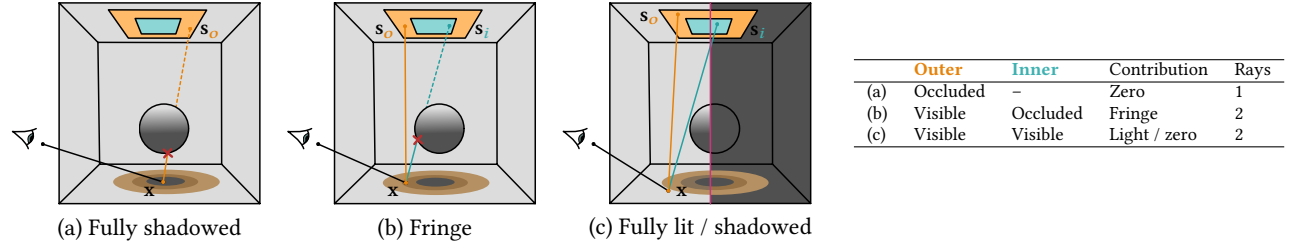


Figure 3: Our algorithm traces up to two shadow rays to sample fringe lights. If the first sample on the **outer** light source is occluded (a), the surface point is treated as shadowed, and no additional ray is required. Otherwise, we sample the **inner** light source, and trace a second shadow ray. If the second sample is occluded, we conclude that the surface point is partially shadowed (b) in which case we return the fringe color. Conversely, if both samples are visible, we treat the point as fully lit by the regular light color (c) (left half), or fully shadowed to render solely the fringe (c) (right half).

Algorithm 1: Fringe area light sampling

```

 $s_o \leftarrow \text{sampleRandomPos}(r_o)$ 
if visible( $x, s_o$ ) then
   $s_i \leftarrow \text{sampleRandomPos}(r_i)$ 
  if not visible( $x, s_i$ ) then
    return evalFringe( $x, s_o$ )
  else
    return evalLight( $x, s_o$ )
return zero

```

Distant and spot fringe lights work analogous to area lights, but require sampling directions with respect to the emission angle instead of positions, as summarized in Algorithm 2.

Algorithm 2: Fringe directional light sampling

```

 $\omega_o \leftarrow \text{sampleRandomDir}(\theta_o)$ 
if visible( $x, \omega_o$ ) then
   $\omega_i \leftarrow \text{sampleRandomDir}(\theta_i)$ 
  if not visible( $x, \omega_i$ ) then
    return evalFringe( $x, \omega_o$ )
  else
    return evalLight( $x, \omega_o$ )
return zero

```

Integration of fringe lights. Fringe lights can be easily added to USD-based pipelines by defining a few additional parameters in custom light api schemas. Generic fringe lights that support both fringe color and light color can be specified with four additional parameters:

- a boolean to enable/disable the fringe,
- a scaling factor to vary the size of the shadow fringe,
- a fringe color, and
- a fringe intensity.

During the production of Netflix’s Leo, artists needed to separate fully lit areas from fringe areas in separate light groups for further post-processing. For that reason, we defined a *fringe-only* mode, that only requires two additional parameters:



Figure 4: Our fringe lights act like any other light source but only illuminate partially shadowed areas. As any other light source, they can be linked to scene geometry. Here, one area light illuminates the whole scene while two fringe lights (same size and intensity, scale factor $m = 1$) create the blue and orange fringe for the outer two logos by shadow linking them to the respective geometry.

- a boolean that turns a light into a fringe light, and
- a scaling factor to vary the size of the shadow fringe.

This means that we could reuse the color and intensity defined for any light source to define the fringe color and intensity instead.

Limitations. At the moment, our implementation does not support colored shadow fringes on dome/environment lights, as it is not straightforward to define a scaling for such a light.

Our usage of fringe-only lights during the production of Netflix’s Leo also meant that the light rig contained two light sources, one main light with fringe disabled and one fringe-only light, incurring a small overhead during rendering. An alternative solution would be to configure our fringe lights to automatically separate fringe and light contributions in two different AOVs, but requires a slightly different workflow than with light groups and more changes to the renderer to configure the additional output channels.

Our solution greatly contributed to the look of Netflix’s Leo and provided artists with intuitive parameters, as we will discuss next.

3 Results and Discussion

Adding fringe lights to the renderer allows lighting artists to get direct visual feedback in (preview-) renders while shaping the look

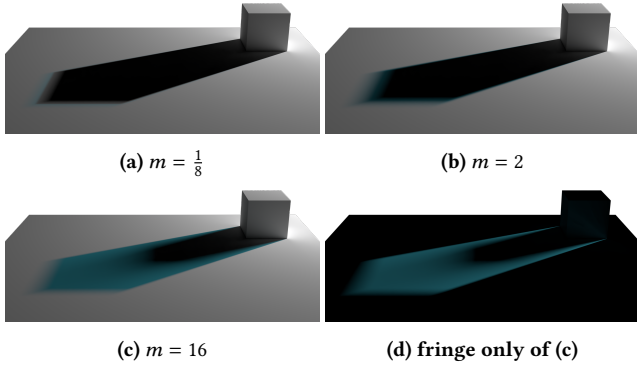


Figure 5: The scale factor m adjusts the size of the shadow fringe. The inner light source visibility is used to switch the contribution of the fringe light, hence the shadow fringe is always limited by the shadow of the inner light source. While (a)-(c) show the fringe light in combination with a regular light source in the same position and with identical brightness, (d) visualizes the fringe light isolated.



Figure 6: Our colored fringe lights were used on Netflix’s Leo for most daylight shots, both outdoor (top) and indoor (bottom). © 2023 Netflix US, LLC. All Rights Reserved.

of the stylized penumbra. As fringe lights act like any other light source in the renderer, they can be rendered together with the rest of the scene and do not require a separate rendering pass. Writing the contribution to a separate light group output allows for further processing during compositing.

Artists can light- or shadow-link geometry to specific fringe lights and define a color for each, leaving fully lit parts and core shadow unaffected (see Figure 4). The scaling factor m can be varied to change the size of the colored fringe, as shown in Figure 5.

Table 1: We evaluate the per sample overhead of using fringe lights for the production scene (Figure 1) and a simple scene (Figure 5 (c)). As a reference, we provide the render time without fringe light (left) and compare the render time per sample to using a combined fringe light (middle) and to using both, normal and fringe-only lights (right).

	No fringe	No primary	All lights
Figure 1	2705.1ms	2940.2ms (+8.7%)	2943.2ms (+8.8%)
Figure 5 (c)	117.4ms	118.9ms (+1.3%)	130.9ms (+11.5%)

Note that for $m < 1$ the fringe light only affects part of the penumbra region, leading to an unrealistic result. Our fringe lights have been used extensively in Netflix’s Leo environments, adding an interesting visual effect to most sunlight shots (Figure 1, Figure 6).

Naturally, tracing additional shadow rays introduces an overhead, as shown in Table 1. In the simple scene (second row), turning a normal light into a fringe light adds an overhead of 1.3% while adding a fringe-only light to the normal light increases render time by 11.5%. Note that, in this case, we doubled the number of light sources and (approximately) tripled the number of shadow rays while the cost of material evaluation and primary ray intersections are minimal. In the production scene on the other hand (first row), tracing additional shadow rays is more expensive due to geometry complexity, hence, adding 8.7% to the render time when replacing the sun light with a fringe light. The additional overhead of using both a fringe-only and a normal light is negligible in this case.

4 Conclusion

We presented our novel stochastic algorithm to integrate colored shadow fringes into our production renderer Glimpse. Integration into the light source sampling and evaluation enables an artist-friendly workflow to alter the look of the penumbra without influencing the lighting of the remaining scene. Compared to shader- or compositing-solutions our approach is more robust and requires no additional manual work apart from the lighting artist defining the look of the penumbra for a light source if desired. We demonstrated the workflow and parameters available to the lighting artist as well as looks that can be achieved with this technique. Production examples show its practical use in one of our most recent shows.

Acknowledgments

We would like to thank Zane Smedley for his help with production resources, and Steve Agland and Rodrigo Janz for their assistance with production material. We would also like to thank our colleagues at Animal Logic for reviewing early versions of the paper and for their support throughout this project.

References

- Chris Brejon. 2020. *CG Cinematography*. <https://chrisbrejon.com/cg-cinematography/>
- David H Foster. 2011. Color constancy. *Vision research* 51, 7.
- Jean-Marc Hasenfratz, Marc Lapiere, Nicolas Holzschuch, and François X Sillion. 2003. A survey of real-time soft shadows algorithms. *Computer Graphics Forum* 22, 4.
- Max Liani. 2023. Tech demo. <https://x.com/maxliani/status/1708371219488403784>.
- Leif Pedersen and Dylan Sisson. 2023. Stylization at Pixar. <https://renderman.pixar.com/stories/stylization-at-pixar/>
- EC3D studio. 2023. Shadow Fringe Blender plugin. <https://ec3dstudio.gumroad.com/l/shadowfringe>.